

The Scenario Controller — Architecture & Design Report

Project: STING Bioinformatics (VR Nursing / Informatics Training) **Engine:** Unity 6000.4.9f1 (URP · OpenXR · Meta XR) · Interaction via **BNG**
VRIF Assembly: **Assembly-CSharp** · global namespace · **Assets/Scripts/Scenario/** **Date:** 2026-07-22

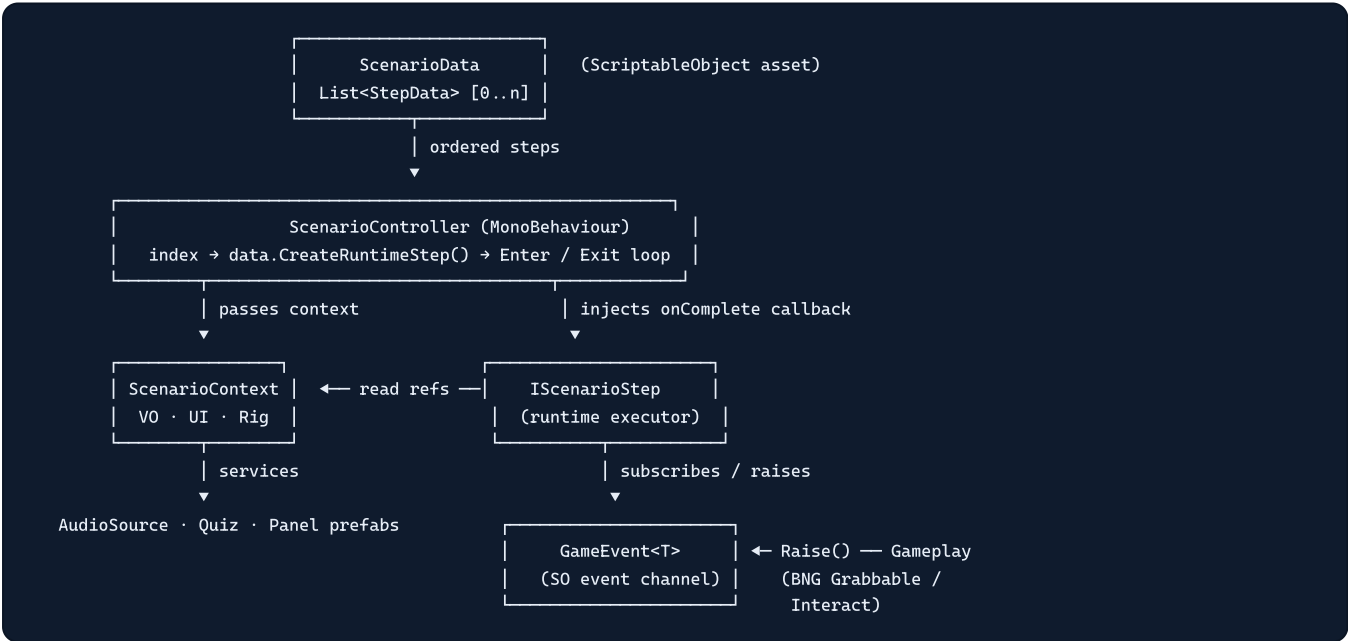
1. Executive Summary

The **Scenario Controller** is a *data-oriented linear sequencer* that drives narrated VR training scenarios from designer-authored assets. A scenario is an ordered list of **steps** (narration, a PC question, a scene event, or "wait until the player does X"). The controller walks the list one step at a time; each step signals when it is done, and the controller moves on. It fires a single "scenario complete" event at the end.

The system is built so that **designers assemble and reorder scenarios entirely in the Unity Inspector**, with no code changes, while **engineers add new capabilities by writing one small class** — never by editing the controller. Communication with gameplay and UI is fully **event-driven** (no polling), and **UI panels are prefabs** that a step spawns and tears down within its own lifecycle.

2. System Overview

Piece	Kind	One-line responsibility
<code>ScenarioData</code>	ScriptableObject asset	The designer-editable scenario: an ordered <code>List<ScenarioStepData></code> .
<code>ScenarioStepData</code>	abstract ScriptableObject	Base for every step <i>asset</i> ; holds only that step's data.
<code>IScenarioStep</code>	runtime interface	The step's <i>behavior</i> : <code>Enter(ctx, onComplete)</code> / <code>Exit()</code> .
<code>ScenarioController</code>	MonoBehaviour	Runs the list: enter → wait for completion → exit → advance.
<code>ScenarioContext</code>	serialized class	Shared references/services handed to every step (VO, UI, rig).
<code>GameEvent</code> / <code>GameEvent<T></code>	ScriptableObject channel	Observer signaling between gameplay and steps.
Panel prefabs	Prefabs	Self-contained UI shown per step (question panel, PC screen, ...).



Control loop (in words): `ScenarioController` holds an `index`. It asks the current `ScenarioStepData` to build its runtime executor (`CreateRuntimeStep()`), calls `Enter(context, onComplete)`, and does nothing else. When the step invokes `onComplete`, the controller calls `Exit()` on that step, increments the index, and enters the next — or fires `onScenarioComplete` if the list is exhausted. **The controller never references audio, UI, or gameplay types.**

3. UI Panels as Prefabs

Every player-facing panel — the PC question screen, informational overlays, the results screen — is authored as an **independent prefab**, not baked into a permanent scene canvas. A step *owns* the panel it needs: it **spawns the panel when it begins and disposes of it when it ends**, so a panel exists only while its step is active.

Lifecycle mapping

Step phase	Panel action
<code>Enter()</code>	Instantiate (or show) the step's panel prefab; wire its callbacks.
running	Panel handles player input and reports back through its callback/event.
<code>Exit()</code>	Unsubscribe callbacks, then hide/destroy the panel instance.

Why prefabs (not one always-on canvas):

- **Scenes stay lean** — no dormant, overlapping canvases cluttering the hierarchy.
- **Self-contained & reusable** — a panel prefab carries its own layout, styling, and script; it can be reused across scenarios and scenes and edited in one place.
- **Lifecycle safety** — a panel cannot linger after its step ends, because its existence is bound to the step's `Enter` / `Exit`. This directly reinforces the "always unsubscribe" rule.
- **Parallel authoring** — a UI designer can iterate on a panel prefab while an engineer works on the step logic, with no merge collisions on the scene file.

This is the **Prototype pattern** in Unity terms: the prefab is the prototype, cloned on demand. In the current build the question step reuses the existing `Quiz` panel toggled via `ScenarioContext.PcUiRoot.SetActive( ... )`; the prefab-spawn model above is the direction each step's panel management evolves toward, with the same `Enter` / `Exit` seams already in place.

4. Design Patterns Used

The system is a deliberate composition of well-known patterns. Each was chosen to remove a specific kind of coupling.

Pattern	Where it lives	What it buys us
Strategy	<code>IScenarioStep</code> + concrete steps	The controller runs against one interface; each step is an interchangeable algorithm.
Factory Method	<code>ScenarioStepData.CreateRuntimeStep()</code>	The data asset decides which runtime class to build, so the controller never <code>switch</code> es on type.
Type Object / Data–Behavior split	<code>ScenarioStepData</code> (data) vs <code>IScenarioStep</code> (behavior)	Designers create "types of step" as assets; behavior stays in code. One data asset can spawn many runtime instances.
Observer	<code>GameEvent</code> / <code>GameEvent<T></code> (Subscribe/Unsubscribe/Raise)	Gameplay raises events knowing nothing about who listens; steps observe without polling.
State (linear)	<code>Enter()</code> / <code>Exit()</code> step lifecycle	Each step behaves like a state with clean entry/exit; the controller is a minimal linear state machine (mirrors the project's EHR state pattern).
Command	<code>InvokeSceneEventStep</code> + <code>UnityEvent</code> / channel	A step encapsulates "invoke this action" and triggers it without knowing the receiver.
Context Object / Mediator	<code>ScenarioContext</code>	A single hand-off of shared services (VO, UI, rig); steps never reach into singletons or <code>FindObjectOfType</code> .
Dependency Injection	<code>Enter(ctx, onComplete)</code> , <code>context.Runner = this</code>	Steps receive their dependencies and their continuation; they never fetch them. Inversion of control.
Continuation-passing (callback)	<code>Action onComplete</code>	Asynchronous completion (VO finishing, an answer arriving, an item grabbed) is expressed as a callback, not an <code>Update()</code> poll.
Prototype	Panel prefabs spawned per step	UI is cloned from prototypes on demand and disposed with the step.
Adapter	<code>TaskEventRaiser</code> , <code>SceneEventRelay</code>	Bridge Unity's <code>UnityEvent</code> world to the <code>GameEvent</code> channel world in both directions.
ScriptableObject Architecture	data, steps, and event channels are all <i>assets</i>	The overarching Unity pattern (Ryan Hipple / "Unite") — configuration and wiring live as project assets, not hard-coded references.

5. Benefits

For designers / content authors

- **Author scenarios with zero code.** Create step assets, drop them into a `ScenarioData` list, reorder by dragging. The sequence *is* the data.
- **Reuse everything.** The same narration clip, question, event channel, or panel prefab can appear in many scenarios.
- **Safe experimentation.** Reordering or swapping steps cannot break compilation.

For engineers

- **Open/Closed by construction.** New behavior = one new `ScenarioStepData` + one `IScenarioStep` . The controller, context, and existing steps are never touched.
- **Low coupling, high testability.** The controller depends only on an interface; steps depend only on the context; gameplay depends only on an event channel. Each layer can be reasoned about — and tested — in isolation.
- **No hidden globals.** Dependencies arrive through `ScenarioContext` , so a step's needs are explicit and visible.

For the runtime / product

- **Event-driven, not poll-driven.** No `Update()` loops spinning on state; steps sleep until an event or callback wakes them — cleaner and cheaper.
- **Deterministic, single-path completion.** Each step completes exactly once; the sequence advances exactly once; the scenario ends exactly once.

- **One audio path.** All voice-over (narration *and* answer feedback) flows through `ScenarioContext.PlayVoice`, so VO behaves consistently everywhere and routes through the Narration mixer group.

6. Worked Example — Control & Event Flow

A four-step scenario: **Narrate intro** → **Ask a question** → **Wait for the player to grab the scanner** → **Trigger a scene animation**.

1. **NarratorStep** — `Enter` calls `ctx.PlayVoice(introClip, onComplete)`. The clip plays; when it ends, the callback fires `onComplete`. Controller: `Exit` → advance.
2. **UIQuestionStep** — `Enter` shows the question panel and subscribes to `Quiz.AnswerSelected`. The player answers → the step unsubscribes immediately (re-entrancy guard), validates the index against `QuestionSO.GetCorrectAnswer()`, plays the matching feedback VO through the same audio path, then completes (correct) or re-asks (wrong, tries remaining) per the retry policy.
3. **WaitForTaskStep** — `Enter` subscribes to a `StringGameEvent` channel. The BNG `Grabbable` on the scanner has its grab `UnityEvent` wired to `StringGameEvent.RaiseString("scanner")` (or a `TaskEventRaiser`). When raised with the matching id, the step unsubscribes and completes. **The scanner knows nothing about the scenario.**
4. **InvokeSceneEventStep** — `Enter` raises an "invoke" channel; a `SceneEventRelay` in the scene runs the scene-bound animation via its `UnityEvent`. With `waitForExternalCompletion`, the step waits on a completion channel the animation raises when finished; otherwise it completes immediately. Then `onScenarioComplete` fires.

7. Extending the System (Open/Closed in practice)

To add, say, a "wait N seconds" step:

1. Create `TimerStepData : ScenarioStepData` with a `float seconds` field and `CreateRuntimeStep() ⇒ new TimerStep(this)`.
2. Create `TimerStep : IScenarioStep` — in `Enter`, start a coroutine on `ctx.Runner` that waits and then calls `onComplete`; in `Exit`, stop it.
3. Add `[CreateAssetMenu(menuName = "Scenario/Steps/Timer")]`.

No change to `ScenarioController`, `ScenarioContext`, or any other step. That is the payoff of the Strategy + Factory-Method + Type-Object combination.

8. Robustness & Safeguards

Risk	Safeguard in the design
A step completing twice (double-advance)	<code>stepCompletedLatch</code> in the controller; <code>completed</code> flag in <code>UIQuestionStep</code> .
Leaked event handlers on restart / re-entry	Every step unsubscribes in both its handler and <code>Exit()</code> ; <code>Begin()</code> exits any running step first.
Fast double-answer skipping/consuming a try twice	<code>UIQuestionStep</code> unsubscribes and locks buttons the instant an answer arrives; re-subscribes only when the question is re-shown.
A stale VO callback firing after interruption	<code>ScenarioContext.PlayVoice</code> cancels any pending wait coroutine before starting a new one; <code>StopVoice</code> cancels it on <code>Exit</code> .
Busy-wait CPU cost	No <code>Update()</code> polling — everything is callback/observer driven.
Stale subscriptions surviving play-mode reload	<code>GameEvent.OnDisable()</code> clears its listener list.

9. Trade-offs & Non-Goals

- **Linear by design.** No branching graphs, save/resume, or custom editor tooling — deliberately omitted to keep the system small and predictable. If branching is needed later, it belongs in a separate layer, not bolted onto this controller.
- **Synchronous completion recurses.** A step that completes *inside* `Enter` (e.g. a fire-and-forget scene event) advances the sequence via the call stack. This is bounded by the step count and is harmless at realistic scenario sizes.
- **ScriptableObject `UnityEvent` cannot bind scene objects.** This is a Unity limitation, not a design flaw. `InvokeSceneEventStep` therefore routes scene calls through a `SceneEventRelay` (an in-scene component whose `UnityEvent` can target scene methods), driven by a `GameEvent` channel.

10. File Manifest

```
Assets/Scripts/Scenario/  
├─ Core/  
│   ├── IScenarioStep.cs           interface: Enter / Exit  
│   ├── ScenarioContext.cs        shared refs + single VO audio path  
│   ├── ScenarioStepData.cs       abstract SO base (CreateRuntimeStep)  
│   ├── ScenarioData.cs           ordered List<ScenarioStepData> asset  
│   └─ ScenarioController.cs      linear runner + completion latch  
├─ Events/  
│   ├── GameEvent.cs             parameterless observer channel  
│   ├── GameEventGeneric.cs      GameEvent<T> abstract  
│   ├── StringGameEvent.cs       typed channel (task/item id)  
│   ├── SceneEventRelay.cs        channel → scene-bound UnityEvent  
│   └─ TaskEventRaiser.cs         gameplay → channel convenience raiser  
└─ Steps/  
    ├── NarratorStepData.cs       VO clip; completes on clip end  
    ├── InvokeSceneEventStepData.cs invoke + optional wait-for-completion  
    ├── UIQuestionStepData.cs      question + validation + retry policy  
    └─ WaitForTaskStepData.cs      completes when a typed task is raised
```

Plus one minimal, backward-compatible edit to `Assets/Scripts/Quiz.cs`: a public `event Action<int> AnswerSelected` so steps can subscribe without a hard manager dependency.

11. Pattern → Benefit Summary

Pattern	Concrete payoff in this project
Strategy + Factory Method + Type Object	Add step types without editing the controller; designers create step "types" as assets.
Observer (SO channels)	Gameplay and scenario are fully decoupled; a grabbed item never references the scenario.
Context Object + Dependency Injection	No singletons/ <code>FindObjectOfType</code> inside steps; dependencies are explicit.
Continuation callback + State lifecycle	Clean async flow with deterministic, single-shot completion — no polling.
Prototype (prefab UI)	Panels are reusable, self-contained, and disposed with their step.
ScriptableObject Architecture	The whole scenario — sequence, data, and wiring — lives as editable assets.